Managing Risk in Multi-node Automation of Endpoint Management

Sai Zeng, Constantin Adam, Fred Wu, Shang Guo, Yaoping Ruan
IBM T. J. Watson Research Center

Cashchakanithara Venugopal, Rajeev Puri
IBM Global Technology Services

***Abstract*— Endpoint management, including patching, health checking, configuration etc., is a key function for data center and cloud management. Managing multiple nodes through automation tools or scripts significantly increases efficiency. However, the risks of adverse impact due to excessive privilege or human error may propagate to a large pool of endpoints and lead to massive service disruptions and SLA (Service Level Agreement) violations. In this paper, we present a system that proactively and systematically manages the risk throughout the lifecycle phases of automation. We present a prototype implementation consisting of an authorization mechanism that guarantees the right level of eligibility and privilege of accessing the automation content (during the deployment stage), and an execution validator that controls the risk of human error which may cause massive damage to the infrastructure (during execution of the automation content). Our current implementation has been deployed to more than a dozen customer environments and achieved an efficiency gain of 58% with high execution accuracy.**

Keywords—multi-node automation, endpoint management, task automation, data center management, cloud management, human error.

I. INTRODUCTION

Endpoint management, which includes provisioning, patching, capacity and resource management, backup and recovery, user id management, compliance and audit tasks etc., is a major expense for modern enterprises. In spite of the trends of increasing endpoint numbers, complexity, and security exposures, enterprises and service providers are being continuously challenged to reduce management cost, control risk, and meet higher service level commitments. Today, enterprises and IT service providers are diligently improving endpoint management processes, operations, and tooling in order to survive and thrive in a competitive market.

Several approaches are being applied today in order to achieve these goals. Among these, process automation (full or partial) can significantly reduce human effort. The key drivers for process automation are data integration, human operations reduction, a knowledge base to replace or accelerate human decision making, etc. Process re-engineering is another popular approach, which involves simplifying and re-organizing process steps. Re-engineering a process makes it leaner by eliminating unnecessary operations, and increases its efficiency by removing bottlenecks and idle time [1]. Labor arbitrage is often used by enterprises with global delivery capacity, where selected operations are performed in low cost areas. Cloud computing also brings opportunities by simplifying the management targets and standardizing the management processes, thereby driving down management cost.

Though using scripts for batch processing has been a common practice, the emerging approach is to use more sophisticated tools to manage a pool of endpoints. In this approach, the manual effort spent performing an action on a pool of endpoints is hardly more than the effort spent on a single endpoint. The productivity savings for such operations is almost proportional to the number of endpoints in the pool. Taking patch management as an example, the traditional way is to login to each endpoint, download the patch, execute the patch, and validate the results, either manually or via a script. Tools like TEM (Tivoli Endpoint Manager) [2], allow an SA (System Administrator) to select multiple nodes in one operation and apply the patch to them simultaneously. TEM pushes the patch to each endpoint in parallel, and executes and validates the patch deployment automatically.

In addition to parallel execution, the second important perspective is repository of reusable automation content. Based on its source, automation can be classified in two categories: *prescribed automation* – provided by tools and/or solutions, and *custom automation* – achieved through tool customization, or leveraging locally developed native scripts, commands, or programs. We view custom automation as the primary driver to address today's manual tasks, which dominate endpoint management costs. Companies that manage complex and heterogeneous IT infrastructures serving multiple customers require diverse management functions to manage customers' legacy environments and to meet customer-specific and regulatory requirements. Those management functions are usually fulfilled by customized content which can be derived from existing scripts or knowledge created by experienced system administrators.

If we make those custom contents available on a large pool of endpoints through parallel execution, the solution can revolutionize how server management tasks are performed. The productivity gain can easily exceed that of traditional automation solutions on the market today. However, such gain does not come without a price. For example, state-of-the-art open-source configuration management solutions like CFEngine [4], Puppet [5], or Chef [6], or commercial systems like TEM [2] allow SAs to perform virtually any operation on a large set of endpoints at once. However, these systems usually do not have any risk control mechanisms that would prevent erroneous or unauthorized operations from being executed on a large number of endpoints. If such operations are not isolated and contained within a limited set of endpoints, they can easily

978-1-4799-0913-1/14/$31.00 ©2014 IEEE

lead to system failures and SLA violations. For instance, a SA can shutdown thousands of production servers by accidentally sending a shutdown command to all the managed endpoints.

Such risk is primarily driven by human error. Human error in system maintenance has been acknowledged as a pressing problem in [7]. In their work, the authors classify the errors into six different categories: operating, assembly, design, inspection, installation and maintenance, and provide a survey of such errors and their impact for several industries: aviation, nuclear power, chemical processing, medical devices and mining. Even though this survey does not include the IT industry, identified human errors are prevalent here as well.

Much research on reliable systems focuses on performance and graceful degradation of the service in the face of failures. VL2 [8], and Autopilot [9] are examples of data center design following these objectives. While this work attempts to build an automated system that can minimize the impact of a failure on the performance of a data center, it does not look into the problem of prevention of human errors due to use of parallel execution features for their daily operations.

Open-source configuration management software ([4], [5], [6], [10], [11]) introduces automation as the means for large scale system management. These systems provide a repository with a set of core system administration libraries that can be extended by the users. Some of these solutions do not provide any additional checks on the correctness of the changes to be applied. CFEngine and Puppet go one step further in this direction. They use resource dependency graphs that are built from the causal dependencies between CFEngine promises, or Puppet resource declarations, respectively. These graphs, used in conjunction with executing scripts in preview, or dry-run mode, are subsequently used to detect errors, such as conflicting changes made to the same resource, or duplicate resource declarations. In addition, CFEngine and Puppet provide platform functions to allow customization of automation content. However, the risk management functions available are restricted to out-of-the-box automation, not the custom automation. Other research architectures ([12],[13]) enable and optimize remote command and application execution through one user interface similar to our approach, but their focus is on optimizing performance, such as execution time, rather than on prevention of execution of wrong or unproven code on a large number of nodes, as in our case.

In Internet security and vulnerability management areas, efforts have been spent on preventing intrusion through ad-hoc execution of injected scripts ([14], [15], [16]). These solutions control command (e.q. SQL, OS) execution on the targets through whitelists and/or blacklists. The syntax validation algorithms vary for each use case to deal with different types of commands, their syntax, policies, and business rules. Our solution framework is different because it applies the white list validation concept to a new environment – that of enterprise multi-node endpoint management.

In this paper, we present a framework to proactively manage the risks introduced by parallel execution of custom automation content. Even though the framework is implemented in TEM, we believe the concept and design can be applied to other tools like CFEngine, Puppet or Chef to achieve higher productivity gain with lower risk.

The rest of this paper is organized as follows. Section II provides an overview of the technology and business risks introduced by parallel execution of custom automation content. In Section III, we summarize our risk management framework. Detailed implementation of authorization and authentication mechanisms for deployment and a set of tools that guard against improperly running an intrusive action during the execution phase are explained in Sections III and IV respectively. Finally we illustrate the efficacy of the current framework solution, and highlight future research directions in Sections V and VI.

II. Overview of Risk Management Framework

System Administrators can cause harm to managed endpoints either inadvertently or through malicious actions. To some extent, role-based access control to scripts and commands can reduce both types of risk by restricting the most destructive scripts/commands to the most experienced and trusted workers. Regardless of the nature of the damage, intentional or inadvertent, the risk management framework must aim to minimize the likelihood of such damage.

There are several types of risks that are managed by the framework:

1) Harmful operations, such as shutdown, reboot. When an SA has access to the command line interface (or equivalent) on an endpoint, there is a risk that a destructive command may be issued by mistake. For example, an SA seeking to learn the time of last reboot might intend to type “last | grep reboot”, but inadvertently leave out the “grep” command. Such commands could bring down the endpoint during business hours, potentially causing significant loss of revenue. The SA might issue commands that remove or alter files that affect business applications, causing erroneous results or application outages. Also, an SA might install and execute an automation script without full understanding of the conditions under which that script can be safely used. Tools such as TEM can enable download and execution of scripts/commands on large numbers of endpoints, greatly increasing the risk magnitude.

2) Excessive resource utilization by the management tasks. Even if the command or script causes no permanent change to the endpoint, it might be highly demanding of physical resources, which can contend with resources required by business critical applications, causing performance degradation and even SLA violations.

3) Illegal operations, such as unlicensed utilities, malware, etc. The risk management framework should tightly control the utilities that an SA can install and execute on endpoints, to prevent distribution of potential malware and software products in violation of license agreements.

4) Unauthorized access (customer level access, user role access) to automation content in shared infrastructure and distributed infrastructure. In a shared infrastructure, SAs may be assigned to work on endpoints that belong to a subset of all customers. Unauthorized access by an SA to the endpoints of a customer to which the SA is not assigned would constitute a security violation and possibly violate privacy laws. For the same reason, if an SA is assigned to manage both Customer A's and Customer B's endpoints, the SA should not be authorized to execute automation content intended for customer A on the endpoints belonging to customer B. As mentioned earlier, inexperienced SAs should have limited access to high risk scripts or commands.

The framework we propose aims to manage the risks in the lifecycle phases comprehensively. During development, rigorous software development practices are followed to govern risk. The main purpose of such processes is to make sure that the automation content is correct and compliant with policies, the risk level is evaluated, and the execution signature is captured to provide integrity. During deployment, levels of access control ensure that the right people have the right access to the right content. We achieve this goal by maintaining highly accurate correlations between automation metadata and customer/user/role profiles. Controls in execution provide the last chance to reduce the risk of propagation to a large pool of endpoints. There are multiple mechanisms to control the risks step by step. To start out, preview mode makes the outcome visible prior to execution, which allows an SA to make informed decision on execution. Syntactic validation will check the invocation of scripts/commands against the permitted white list. In addition, execution can be forced to conform to various patterns in order to contain the risk. For instance, instead of allowing issuance of the shutdown command to thousands of servers in one operation, the pattern can apply shutdown to smaller groups of servers in sequence, providing an opportunity to catch and contain errors. Finally runtime governance can enforce a review from a skilled worker prior to execution, which can also help to moderate the risk.

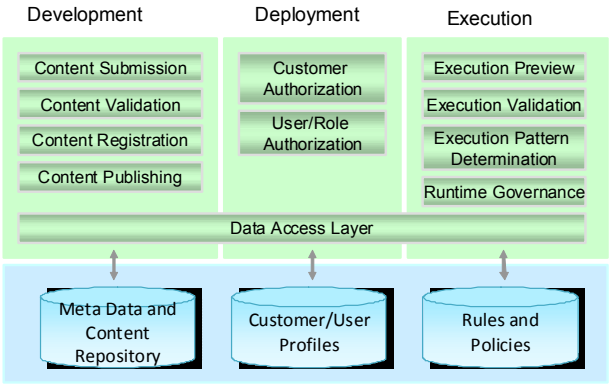

Fig 1.Overall Framework

III. Control the Risk during Custom Automation Deployment

We control risk during the deployment phase by enforcing levels of authorization. There are two levels of authorization: customer level and user role level.

Customer level access determines the eligibility of published automation content. This feature allows flexible customization tailored for the customer environment without introducing risk due to excessive, disallowed, and conflicting automation content. We carefully design the metadata models so that each customer is granted access to, but has flexibility to choose from, automation content for various tasks. For example, a customer contract may grant access to only Operating System automation in the customer's environment. And separation of such decisions can be managed at the customer level through the definition of customer profiles. This is an important feature for service providers who manage data centers for multiple customers on a large scale through shared infrastructure.

User role authorization determines what automation content can be executed on what endpoints based on user role. There are two parameters to classify a role in our current implementation, namely function and risk level. In the strategic outsourcing business, SA responsibilities are defined in fine granularity based on job function and organization, for instance, job responsibility to maintain operating systems, databases, or applications like SAP. With such types of job responsibility, it makes sense to tag the desired automation content with function, and link it to corresponding user role definitions. For instance, if an automation script is meant to get the list of services running in the OS, the script is classified as function 'Operating System', and mapped to role as OSA (Operating System Administrator). If a script was developed to automate database backup, it will be tagged as a 'Database' function, and linked to the DBA (DataBase Administrator) role. In Figure 2, automation content such as a script or a command, can be associated with one or many functions, and functions can be mapped to eligible roles, so that the model can be scaled easily to support a variety of situations.

The second consideration of role definition is based on the risk level of the content. Risk properties are used to characterize the risk level of the automation content, and properties are correlated with the corresponding role definitions. As a simple example, consider SAs that are responsible for Level 1, 2 and 3 support. Skill requirements increase when the level of support increases. The corresponding role definitions can be SAL1, SAL2, and SAL3. A simple way to classify risk level of automation content is by these levels. As a result, SAL1 is eligible for L1 automation content, SAL2 is eligible for L1 and L2 content, and SAL3 can access content of all levels.

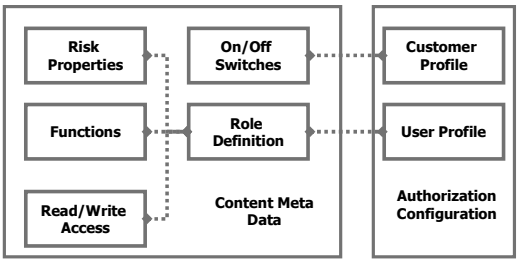

Fig 2. Mapping of Content Metadata and User/Customer Profiles

The user profile defines which roles a user is eligible for, which automatically determines which automation content a user has access to, and what operations a user can perform for eligible automation content. A user can have one or many roles based on his or her job functions, skill level and intended operations.

Configuring customer profiles and user profiles is a one-time task during deployment, and grants access for the right user to the right customer and the right content, and avoids the risk of excessive authorization privilege.

IV. Control the Risk During Custom Automation Execution

Our framework provides a large set of automation scripts, which have been reviewed and published. We also allow users to input individual commands via a text input box. During execution, we stage the protective procedures into three stages. When a management task comes in, first we search for an approved script that is suitable for the task. If no match is found, or the script is not sufficient to perform all the actions in the task, an SA will manually input commands via our management tool interface, which will perform validation and preview functions. Finally, if the system has to be accessed via native tools, protection will rely on the protected shell on the managed node.

We have developed a syntactical validator to review the user input against a set of pre-approved non-intrusive commands which we call "white list commands." For intrusive commands, we use a similar filtering approach but provide an additional alert or simulate and preview mode to increase user awareness, plus additional control mechanisms like execution pattern and protected shell which we will discuss later. We compiled the list by reviewing command suggestions submitted by the internal system administrator community, and implemented a process for inclusion of new additions.

The syntactical validator verifies that the user input on the command line complies with a set of rules. Rules specify which commands or scripts are allowed for each user group, as well as which parameters are allowed for each command. The validation rules reduce the risk of a non-expert user running improperly an intrusive action (e.g., restrict the options for '*/sbin/fdisk*' to '*–l*' and '*–u*' in order to only view, not change the disk partition tables), or launching an action that consumes significant resources on a large number of nodes (e.g., exclude the option '*–c*' for '*/bin/netstat*', to avoid producing network statistics for each node every second). A command line is distributed to the endpoints for execution only after it passes the validation check.

The parameters are validated only for commands, and not for scripts. We assume that scripts are validated during the development governance process, which mandates that the scripts incorporate functions that check and validate the input parameters. TABLE I. summarizes the parameter policies that have been defined for commands. The reason we do not attempt to validate automation scripts themselves is that the logic in scripts can be complex, and commands that might ordinarily be restricted can be permitted when embedded in the proper logical sequence.

TABLE I. Example of Validation Policies for Command Parameters

Commands with no argument restrictions:
/usr/bin/du, /usr/bin/ls, /usr/bin/uptime
Commands that are safe with no arguments:
/usr/bin/alias, /usr/bin/hostname
Commands with restrictions on arguments (set of arguments allowed):
/sbin/fdisk; only -l, -u, -c
Commands with restrictions on arguments (set of arguments not allowed):
/bin/netstat; except -c
Commands with restrictions on arguments (set of arguments required):
/usr/sbin/instfix; require –i

Some instructions can use regular expressions as input. This complicates the validation process, because the meaning of a character can be different inside and outside of a regular expression. For example, consider the AIX command */usr/bin/df –g | /usr/bin/egrep –y '8[0-9]{1}%|9[0-9]{2}%|100%'* that prints all the disk partitions for which the usage is 80% or higher. The first pipe character ('|') is outside a regular expression, and it plays the role of a command separator, as it redirects the output of the *'df'* command to serve as the input of the *'egrep'* command. The second and third pipe ('|') characters act as logical "or" operators inside the regular expression, allowing the selection of disks where the usage is 80-89%, or 90-99%, or 100%.

We apply a language grammar to precisely and flexibly define the command lines that are acceptable by the validator. Part of the Extended Backus-Normal Form (EBNF) notation of the language grammar are presented in TABLE II. (for Linux/Unix; similar specifications are defined for Windows).

TABLE II. EBNF Grammar for User-Entered Command Line in Linux

```
{}* zero or more repetitions; {}+ one or more repetitions; | or
cmdline::=cmdstmt {{separator}+ cmdstmt}*
cmdstmt::=cmdnores|cmdnoarg|cmdres
separator::=\||\\||&|&&|;|)|(
cmdnores::=cmd {{option}* {userinput}*}*
cmdnoarg::=cmd
cmdres::=cmdonly|cmdexcept|cmdrequire
cmdonly::=cmd {onlyoption {userinput}*}+
cmdexcept::=cmd {exceptoption {userinput}*}+
cmdrequire::=cmd {nonrequireoption {userinput}*}*
  {requireoption {userinput}* {nonrequireoption {userinput}*}*}+
cmd::=text|.cmd1|/cmd1|cmdprefix cmd
```

```
cmdprefix::=/bin/sh|/bin/shell|/usr/bin/perl|...
cmd1::=text|.cmd1|/cmd1
onlyoption::=option
exceptoption::=option
nonrequireoption::=option
requireoption::=option
option::=-text|--text
userinput::=text|/userinput|.userinput|regexp
regexp::= ‘{regexp_char}*’
text::={letter|digit|_|"}text1
text1::={text_char}*
letter::=a|b|...|z|A|...|Z
digit::=0|1|...|9
other_char::= ,|=|/|!|*|(|)|[|]|{|}|-|’|"|?|;|.| |:|+|_|%|@|&|#|$|<|>|\|^|‘|˜
all_char::=letter|digit|other_char
regexp_char::=all_char – {`|>|\\}
text_char::=regexp_char – separator
```

The grammar above is used to parse and validate single commands, composite commands, or commands that accept regular expressions.

To validate composite commands, each command line input is tokenized into a series of commands; each command and its set of parameters is checked against the rules described in the white list. If a part of a composite command fails the validation, the entire composite command will be rejected and none of it will execute on the endpoints. For example, consider a Windows composite command such as: '*dsquery user ou=Marketing, dc=Microsoft, dc=com | dsmod group "cn=Marketing Staff, ou=Marketing, dc=microsoft, dc=com" -addmbr & shutdown*'. As *shutdown* is not in the white list, the entire composite command will be rejected, even though the other parts of the command represent legitimate Active Directory operations.

In order to avoid excessive complexity of the validator, a small set of characters are not accepted on the command line. Back ticks (`) and escape characters are excluded from the shell. Even though back ticks can be useful in certain situations, (the path for certain Solaris commands changes between hardware versions, and using back ticks as in /usr/platform/`uname -i`/sbin/prtdiag, allows calling the same command without having to re-write the command for each hardware type) validating such an expression would require spawning a new process that would check the expression between back ticks, and then pass the control back to the main checker. While feasible, this solution would be more complex. A similar problem occurs with escape characters. Regular expressions are contained inside single quotes (and not double quotes), as this prevents expanding system variables at runtime. Finally, right brackets ('>') are forbidden, as they could be used to overwrite a system file by re-directing into it the output of a command (e.g. ls –l > /etc/passwd). If a SA still wishes to use commands with special characters, he or she can wrap the commands in a script file, submit it to the governance process, and use the TEM framework to deliver it to the endpoints for execution. By doing this, we can avoid developing a very complex validation function to cover infrequently used characters, but still support all the commands a SA wishes to run for his or her tasks, as allowed by the governance process.

The next control procedure is to configure the runtime execution pattern. This is especially important for dangerous commands like shutdown, delete, remove, etc. Even a skilled SA will sometimes commit errors. He or she could issue dangerous commands to wrongly selected target servers, or in violation of corporate policies, such as outside allowed change windows, or non-compliant with individual policies on target servers. There are several approaches to address such risks. For instance, by configuring a maximum number of target endpoints for each execution, we can limit the execution of dangerous commands to a smaller group, reducing the impact of erroneous operations and providing the opportunity to suspend subsequent executions if failures or issues are observed. Another potential mechanism is to reject execution if policies are violated. Additionally, a backup and rollback mechanism could be implemented to react to failed operations; this is especially important for servers hosting business critical applications. Another approach is to add peer review as additional screening in the execution process. For instance, when an SA plans to execute an automation task, a peer or approver will review the operation and list of target servers before execution is permitted.

One last procedure to prevent disallowed operations or unwanted changes to a computer system (including the OS and the application stack) involves executing commands and scripts at the endpoint in a protected shell. The protected shell executes an instruction in three steps. First, it performs a syntactical validation of the command names and their parameters, as described earlier in this section. Second, it simulates the execution of the command, and determines the resulting changes to the system configuration. The shell blocks any commands that attempt to change a part of the configuration that is marked as read-only. Third, if the validations performed in the first two steps succeed, the shell executes the script or command.

The validation described in the second step above takes place in a specific context that determines which parts of the configuration can be changed. This context can be built starting from a change ticket, user roles, and/or policies. By dynamically building the read-only configuration, the protected shell limits the system administrators to actions that are appropriate for their role or tasks at hand. For example, during a change window, installation of new patches is allowed, but this permission may be revoked at any other time; or a database administrator might not have access to the configuration of an email server. This context-based validation is the main difference between the protected shell and similar change preview mechanisms, such as the dry-run mode in Puppet [5].

V. Solution Deployment and Adoption

The potential savings from script and command execution across a large pool of endpoints in a single invocation is counterbalanced by the risk of inadvertent or malicious damage from disruptive actions. Broad adoption of the solution requires built-in safety measures and in some cases a process by which customer teams can develop confidence in the safety and

effectiveness of the tool. We decided to prohibit direct invocation of intrusive commands, but to allow execution of carefully tested scripts. Preventing direct invocation of intrusive commands was achieved by a command validator that only permits a set of pre-approved platform-specific commands. Validation complexity was driven by the fact that many useful non-intrusive commands have options or parameters that make the command intrusive; consequently the validator has to parse the full command string. We kept the validation complexity under control by allowing users to input commands that take as input pre-defined options, or a simple form of regular expressions that do not allow expanding system variables or launching other shells. The resulting command set contains top usage patterns/syntax of commands, but is not overly complex. In effect, the allowed commands and options enable only retrieval of information from the endpoint.

To help customer teams build up confidence in intrusive automation scripts, we deploy in two phases. In the first phase, the results of script simulation are shown to the SA, and only after SA approval is the intrusive script actually executed. For example, the disk cleanup operation first scans the disk for files matching pre-configured patterns and presents to the SA the list of files that would be deleted. After the SA edits the list and approves, the intrusive script is executed. Over time, the SME modifies the file matching patterns to conform with customer policy, and the SA finds that there is no need to edit the list. At that point, the SAs are confident in the safety and effectiveness of the script, and the second phase begins; whenever the script is invoked, the disk is cleaned up without review.

From September 2012 to March 2013, we have deployed the solution to more than a dozen customer accounts in a large IT services company. We also observe increasing interest from other accounts in recent months.

We calculate productivity gain by collecting historical execution data such as name of action, set of target endpoints, status of execution, etc. Productivity gain was defined as (n-1)/n, where n is the number of target endpoints for an action. Thus an action executed on only one endpoint has productivity gain of zero. The largest set of endpoints for a single action we have observed is 1661. The overall productivity gain during this period was 58%, greater than that of many other automation solutions. Additionally, the very small number of errors reported by the SA community is indicative of the effectiveness of our rigorous control throughout the entire life cycle. We believe this demonstrates that with careful design, the potential risks of new automation technologies can be minimized while potential business benefits are realized.

VI. FUTURE RESEARCH

We believe endpoint management is still a labor-intensive industry. Our research has revealed a number of future research opportunities which can maximize the automation level of endpoint management, while minimizing the risk of such large scale automation. Some of the related works are already discussed within the paper. In net, the key research and technology challenges are to address the following questions: How do we intelligently check automation content, discover risky syntax embedded in a script, and classify the risk level of the automation content? How do we preview or simulate changes prior to execution? How do we determine the execution patterns which can minimize risk and reduce human involvement?

[1] Ruffa, Stephen A. (2008). Going Lean: How the Best Companies Apply Lean Manufacturing Principles to Shatter Uncertainty, Drive Innovation, and Maximize Profits. AMACOM. ISBN 0-8144-1057-X.

[2] Tivoli Endpoint Manager, http://www-01.ibm.com/software/tivoli/solutions/endpoint/?s_pkg=bfmw

[3] E. Haber and J. Bailey, "Design Guidelines for System Administration Tools Developed through Ethnographic Field Studies," Proceedings of the 2007 symposium on Computer human interaction for the management of information technology, pp. 1

[4] CFEngine - Distributed Configuration Management. http://cfengine.com/

[5] Puppet Labs: IT Automation Software for System Administrators. http://puppetlabs.com/

[6] Chef | Opscode. http://www.opscode.com/chef/

[7] Dhillon, B. S., and Y. Liu. "Human error in maintenance: a review." Journal of Quality in Maintenance Engineering 12.1 (2006): 21-36.

[8] Greenberg, Albert, et al. "VL2: a scalable and flexible data center network." ACM SIGCOMM Computer Communication Review. Vol. 39. No. 4. ACM, 2009.

[9] Isard, Michael. "Autopilot: automatic data center management." ACM SIGOPS Operating Systems Review 41.2 (2007): 60-67.

[10] Salt Stack. http://saltstack.org/

[11] Ansible - SSH-Based Configuration Management & Deployment. http://ansible.github.com/

[12] Claudel, B., Huard, G., & Richard, O. (2009, June). TakTuk, adaptive deployment of remote executions. In Proceedings of the 18th ACM international symposium on High performance distributed computing (pp. 91-100). ACM

[13] Brim, M., Flanery, R., Geist, A., Luethke, B., & Scott, S. (2001). Cluster command and control (c3) tool suite. Parallel and Distributed Computing Practices, 4(4)

[14] Wassermann, G., & Su, Z. (2007, June). Sound and precise analysis of web applications for injection vulnerabilities. In *ACM SIGPLAN Notices* (Vol. 42, No. 6, pp. 32-41). ACM

[15] Vallentin, M., Sommer, R., Lee, J., Leres, C., Paxson, V., & Tierney, B. (2007). The NIDS cluster: Scalable, stateful network intrusion detection on commodity hardware. In *Recent Advances in Intrusion Detection* (pp. 107-126). Springer Berlin/Heidelberg.

[16] Ordinalities, W., & Mappings, T. (2009). CWE-77: Improper Sanitization of Special Elements used in a Command ('Command Injection'). *CWE Version 1.5*, 74

[17] D. Grune, C. J.H. Jacobs, Parsing Techniques: A Practical Guide. Springer, 2nd edition, 2007

[18] De, P. Gupta, M. Madduri, V. & Singh J. (September, 2011). Facade: Collaborative Creation, Peer Review, and Execution of IT Tickets. (2011 : ICSEM)